

Methodology v2: Automated Backend Generation from Triton IR to a Declarative Toy ISA

Audit-hardened revision

SegFault project

September 2026

Abstract

We describe a method for taking a *declarative description of a tensor ISA* and automatically producing a *Triton-IR-consuming backend* for it, registered as a device with PyTorch’s `torch.compile` stack. This revision (v2) replaces v1. Every quantitative claim is tagged with its provenance; every claim in v1 that failed verification is listed in Appendix B together with the correction. The empirical claims about Triton IR were tested against **Triton 3.7.1** and are marked **VERIFIED**. The scope is a *demonstration of the mechanism*, not operator coverage: the deliverable of record is a real `torch.compile` backend, and the method’s transferability is *measured* across two independently authored ISA descriptions rather than asserted. The claim is deliberately narrow and stated in §1.

1 The claim, precisely scoped

Given a declarative description of a tensor ISA, a `torch.compile` backend for it can be generated automatically and registered as a PyTorch device, with measured operation coverage on a graduated kernel corpus and numerical validation against a reference implementation.

Three choices make this claim testable rather than merely true, and all three are in the plan, not in future work. First, the deliverable of record is the *PyTorch seam* — `register_backend` plus a `DeviceInterface` implementation (§3, §11) — not a bespoke harness, because that is the interface a device must satisfy and the exact interface the claim names. Second, the toy ISA is specified with *alternatives and costs* (§4.2), so that **generate** means “select the minimum-cost admissible lowering” and not “fill in a template”. Third, the method is re-applied to a *second*, independently authored ISA and the transfer is measured (§4.5): a method demonstrated only on the ISA it was designed against demonstrates an implementation, not a method.

What the claim does *not* assert. v1 asserted a “missing cell” in the literature. That assertion is correct and survives verification, but it must be stated at the right unit of analysis.

- **Not synthesis.** At Triton’s abstraction level the tiling is already resolved. Our extraction confirms `tensor<64x32xf32> * tensor<32x64xf32> -> tensor<64x64xf32>` and `BM=64, BN=64, BK=32` are *literals in the IR*. ACT must *invent* tiling via equality saturation over partially-instantiated instructions plus integer constraint programming over addressing attributes (ACT §3.1, §5, §6); we inherit it. The claim is therefore about *lowering plumbing*, not about tiling synthesis.
- **Not comparison with ACT or Triton-MTIA.** ACT auto-generates for **XLA-HLO IR** and was evaluated on six accelerator platforms; Triton-MTIA is a production backend built by hand. We do not compete with either.
- **Not a drop-in tool.** The artifact is a Python program operating on frozen `ttir` text. Every reference implementation (ACT, `triton-shared`, `triton-linalg`, `intel-xpu-backend`, Triton-MTIA) is a C++ MLIR pass. Ours does not integrate as a pass and does not claim to.

- **Not coverage.** The corpus is four kernels on a toy ISA with two instruction variants. Coverage numbers are reported per tier and are not comparable to production figures.

Why the cell is nevertheless worth filling. The gap is verified by search and is structurally specific: automated ISA→backend generation exists *for XLA*, and Triton backends exist *built by hand*. The two have not met. The prize is quantifiable:

Surface	Size	Provenance
ATen operator schemas	2,590	aten/src/ATen/native/native_functions.yaml, - func: entries
register_lowering() in lowering.py	209	torch/_inductor/lowering.py
register_lowering() across torch/_inductor	265	repository count
register_decomposition() in torch/_decomp	186	repository count
Core ATen IR surface (subset)	~540	Triton-MTIA §1, citing PyTorch Core ATen IR docs

Five organizations have now hand-built the Triton backend layer: Meta (MTIA), Microsoft (triton-shared), Cambricon (triton-linalg), Intel (intel-xpu-backend-for-triton), and most recently Qualcomm (hexagon-mlir, open-sourced February 2026; Absar et al., arXiv:2602.19762, which ingests Triton IR through hand-written MLIR passes — fusion, TCM tiling, HVX vectorization, DMA double buffering — with no declarative hardware description as input). Meta’s published account of the cost: a compiler developed from the ground up, 16 kernels ported of which 5 required rewriting, and upstream TorchInductor codegen tailored so pointer arithmetic stayed analyzable (Triton-MTIA §3, §5.1, §8.1). Repetition of the same artifact across five organizations is the evidence that automating it has value.

2 Grounding: verified inheritance map

Each source contributes a specific, checkable element. Provenance is stated per row; the section and table numbers have been verified against the source PDFs.

Element inherited	What is borrowed, and what is not	Provenance
Declarative ISA as generator input	Borrowed: a declarative, tensor-level ISA description consumed by an automated generator. Not borrowed: <code>pii</code> machinery and equality-saturation search.	TAIDL (MICRO’25), ACT §3.2, §5
α / β attribute split	Borrowed as a first-class concept. Confirmed to be TAIDL’s own API shape, not merely prose: <code>add_instruction(instruction, computation_attr, addressing_attr, ...)</code> .	<code>taidl/taidl/accelerator.py</code> (lines 37–40)
Compute / memory phase split	Borrowed: instruction selection over tensor operators versus buffer/addressing resolution.	ACT §5, §6
Pipeline shape	Borrowed: canonicalize $\rightarrow$ idiom detection $\rightarrow$ loop/shape reconstruction $\rightarrow$ metadata emission (four phases, eight passes).	TensorLift Table 2, §3
Inversion of loop reconstruction	Borrowed and inverted. TensorLift <i>reconstructs</i> <code>scf.for</code> reductions from flattened bit-level arithmetic; Triton already emits them. Verified: our Tier-1 <code>ttir</code> contains exactly one <code>scf.for</code> with <code>iter_args</code> around exactly one <code>tt.dot</code> . An entire phase becomes recognition.	TensorLift Table 2 passes C6/C7; our extraction
Annotate-don’t-rewrite	Borrowed verbatim. TensorLift: “passes annotate rather than rewrite”.	TensorLift §3.2
Opaque fallback	Borrowed: an instruction matching no template receives no annotation, and emission falls back to opaque semantics rather than producing incorrect output.	TensorLift §3.2
Soundness vocabulary	Borrowed: every operation is either lowered or explicitly flagged; nothing is silently miscompiled.	ACT §3.3
Target compiler shape	Borrowed: Frontend (<code>ttir</code>) $\rightarrow$ Middle-end (map ops to hardware, decide DMA vs. scalar) $\rightarrow$ Backend (architecture lowering) $\rightarrow$ Codegen. We map DMA-vs-scalar onto DMA-vs-unsupported.	Triton-MTIA Fig. 4, §3.3
Evaluation discipline	Borrowed: report per-module/per-tier, report coverage and not only speed, separate compute-dominated from long-tail.	TensorLift Table 3; Triton-MTIA Table 1, §7.1.1, §7.1.3

2.1 Negative reference: corrected

v1 catalogued `triton-shared`’s “documented failure modes” as fragile pointer/mask/offset analysis, crashes on unstructured access, and a monolithic hard-to-extend pass, “cited directly by its own maintainers”. **CORRECTED** Three of the three specific characterisations are unsupported, and the status of the artefact changed. The corrected reference, with evidence:

- **The repository is archived.** `README.md` line 1: “*This repository is no longer maintained.*” Last commit 2025-12-05, “*Update README with maintenance notice (#367)*”. It is not a live competitor and must not be framed as one.
- **“Crashes on unstructured access” is false at HEAD.** An unstructured path exists and is tested: `lib/Conversion/TritonToUnstructured/TritonToUnstructuredPass.cpp`

(595 lines), `lib/Conversion/UnstructuredToMemref/UnstructuredToMemrefPass.cpp` (422 lines), `python/examples/test_modulo.py`, plus a `test/Conversion/TritonToUnstructured/` suite containing gather, vector-add, softmax and matmul cases.

- **The modulo case — our Tier 3 negative control — is explicitly supported upstream.** `include/triton-shared/AnalysisStructured/PtrAnalysis.h` documents handling for `ptr + (row_offsets[:,None] % mod_offset + some_number)` and states that the modulo term *is structured*; the `tts.make_tptr` operation’s `shape` field is defined as “the boundary by which addresses wraps around (constant zero indicates no wrap-around in the corresponding dimension)”. This must be acknowledged as a design choice on our side, not a limitation on theirs.
- **“Monolithic hard-to-extend pass” is unsupported.** `triton_shared.cc` is 8 lines and no file, README or issue makes that claim. The genuine maintainability signal is different and is worth reporting accurately: two parallel pointer analyses coexist — `lib/Analysis/PtrAnalysis.cpp` (1,375 lines) and `lib/AnalysisStructured/PtrAnalysis.cpp` (1,959 lines).
- **What the maintainers actually documented** is narrower and is quotable: “the prototype only supports lowering programs that have structured memory access patterns” and “The analysis is still in its early stage and does not support all scenarios”. Measurable incompleteness is visible in the issue tracker (e.g. #201 `tl.make_block_ptr` lowering, #233 `tt.atomic_rmw` unexpected op in pointer sequence, #243 unsupported `tt.reduce`, #298 `AddPtr` right-hand operand, #309 mask conversion). These are *diagnostics*, not crashes.

Consequence for the design. The scope boundary in §10 remains correct, but its justification changes. We bound the pointer walk because *a fixed hop budget is what makes a five-day demonstration feasible*, not because a reference implementation crashes. The original real-world grounding for Tier 3 is unaffected and is verified: Triton-MTIA §8.1 reports that “the pointer arithmetic code could not be lowered to DMAs because it used modulo operations to wrap a tensor of pointers around the size of the tensor”. Note also that Meta’s *response* was to prevent the pattern upstream by tailoring Inductor codegen to avoid modulo and division (Triton-MTIA §5.1), whereas we detect and fall back. Both are legitimate; they should not be conflated.

3 The integration surface: PyTorch

This section is new in v2. v1 aimed at a bespoke NumPy harness, which is the wrong target: PyTorch already defines the deployment seam, and targeting it converts the artifact from “a program that emits toy assembly” into “a backend that gives `torch.compile` a new device”.

3.1 Two distinct surfaces

A hardware vendor integrating with PyTorch must satisfy *two* independent surfaces. Conflating them overstates the problem.

Surface	Content	Size
Device integration	Device registration, runtime, streams and events, allocator, RNG, serialisation, autocast, distributed ProcessGroup, profiler stubs, autoloader entry point.	Bounded. PyTorch’s own in-tree reference (OpenReg) implements it at <i>stub level</i> .
Compiler backend	A <code>torch.compile</code> backend plus the device abstraction that the compiler uses.	Hard, and this is what we target.
Eager operator library	Per-operator kernel implementations for non-compiled execution and for fallback.	2,590 ATen schemas. This is what the compiled path largely removes.

PyTorch’s in-tree reference implementation states its own scope explicitly (`test/cpp_extensions/open_registration_extension/torch_openreg/README.md`): the goal “is **not** to implement a fully functional, high-performance PyTorch backend, but to serve as a **minimalist reference implementation for mechanism verification**”, and implementations “should be **STUB-level**”. It also emulates a device on CPU (`libopenreg.so`: “A DSO that uses the CPU to emulate a CUDA-like device”).

This is the architecture we adopt. The NumPy interpreter of §14 is kept, but demoted from “the front door” to “the device emulator” — exactly the role `libopenreg.so` plays. It is driven through PyTorch’s real registration APIs.

3.2 The two APIs we register against

```
# torch/_dynamo/backends/registry.py
def register_backend(compiler_fn=None, name=None, tags=()) -> Callable[..., Any]

# torch/_dynamo/device_interface.py (class DeviceInterface begins at line 40)
from torch._dynamo.device_interface import DeviceInterface, register_interface_for_device
```

`DeviceInterface` spans 30 method slots across 4 nested classes (`device`, `Event`, `Stream`, `Worker`). Most have usable defaults; a stubbed implementation is sufficient — PyTorch says so of its own reference. The reference pattern to copy is `test/cpp_extensions/open_registration_extension/torch_openreg/torch_openreg/compiler.py`, which contains both the backend entry point and an `OpenRegInterface(DeviceInterface)` subclass.

Honest caveat. The compiled path does not fully eliminate the operator library; it moves the floor. Triton-MTIA achieved 94.5% (inference) / 94.7% (training) success over unique operator $\times$ dtype $\times$ shape $\times$ non-tensor-argument combinations, yet its Inductor-generated kernels accounted for 50% of layers and 47% of non-GEMM execution time (Triton-MTIA Table 1, §7.2.2). Graph breaks, unsupported operations and eager sections fall back to ATen or CPU. A vendor still needs a fallback floor. *We attack the ceiling, not the floor*, and the write-up says so.

4 Toy ISA description language

The schema is TAIDL-flavoured: not TAIDL itself, but recognisable as its miniature. It is expressed as YAML to keep the assembler simple. Two v1 defects are fixed here.

4.1 Data model

A flat memory space. No scratchpad/accumulator distinction initially: bank and tiling constraints are exactly the complexity TensorLift’s RTL extraction exists to recover, and this project starts downstream of that problem. This is a stated narrowing from TAIDL, whose `add_data_model(model_name, dimensions, unit_dim, var_type)` carries dimensionality and unit dimensions.

4.2 Instructions, with alternatives and costs

This is the most important correction in this section. **CORRECTED** v1 specified *one* DMA, *one* MAC and *one* elementwise instruction. That schema contains **no alternatives**, so the generator has nothing to choose and degenerates into a template filler. A generator that never chooses is not a generator. The schema therefore specifies:

Instruction	Kind and addressing	Admissibility constraint	Cost
DMA1D	memory; 1-D contiguous, unit stride	none beyond in-bounds	1.00 / word
DMA2D	memory; 2-D tiled, write-combining	<code>shape[0]</code> and <code>stride[0]</code> divisible by 4	0.60 / word
MAC8	compute; 8×8 outer-product tile	both operands 4-word aligned	1.00 / MAC
MAC16	compute; 16×16 tile	operand rows contiguous: <code>stride[1] == 1</code>	0.70 / MAC
EPI	compute; elementwise add / relu, post-loop	accumulator dtype equals operand dtype	0.50 / element

Two DMA instructions and two MAC instructions, differing in addressing, tile shape and cost, plus one elementwise instruction for the epilogue step. Five, not three — and the difference is the point: **a generator with no alternatives has nothing to decide.**

constraints is kept, narrowly. Each row carries a decidable admissibility predicate over tile shape, stride and alignment. This is the part of TAIDL’s API that matters most here, and it is not cosmetic: a constraint is what makes a lowering *inadmissible* for a given operand, which is what makes selection succeed or fail rather than be a table lookup. TAIDL’s own `add_instruction(instruction, computation_attr, addressing_attr, cost, update, constraints)` takes `cost` and `constraints` as parameters; the schema drops only `update` (microarchitectural state, which presupposes a semantics language this project does not have). With alternatives, admissibility predicates and costs, the generator’s task is ACT §7’s formulation exactly: enumerate the admissible lowerings, take the minimum-cost one.

Accumulation semantics belong in the schema. Each MAC row declares its accumulator precision, its reduce dimension and its *accumulation order* (sequential, *k*-major). This is not bookkeeping. Floating-point addition is not associative, so an emulator that reduces in a different order from the reference produces different rounding; without a declared order the tolerance of §14.2 is unfalsifiable, because any mismatch can be blamed on reassociation. Declaring the order makes the two comparable, or makes the tolerance derivable, which is the standard the precision section sets.

Fidelity caveat. v1 claimed the schema to be “structurally recognizable” as TAIDL’s miniature. That is generous. TAIDL is a Python DSL with an ANTLR4 grammar (`taidl/antlr4/`

IDLV2Parser.py), `set_inputs/set_outputs` addressing expressions, a semantics language with `ENTRY` and `ROOT` forms, and `update` and `constraints` mechanisms carrying microarchitectural state. Our YAML captures the α/β split and per-instruction cost; it drops `update` and `constraints`. The write-up says so rather than implying parity.

4.3 Explicit non-goals

Non-goal	Corresponding full-generality feature
No saturation / clamp semantics	TensorLift Table 2 pass B5 (<code>detect-clamp</code>); TAIDL fixed-point types
No multi-bank memory	TensorLift recovered multi-bank DMA configuration from RTL
No control-flow instructions beyond the fixed reduction pattern	ACT §6 inner-loop tiling and padding
No loop primitive in the ISA	Consistent with TAIDL: instructions are per-tile; loop tiling is the compiler’s job (ACT delegates it to XLA’s autotuner, ACT §8.6)
Greedy matching, no search or backtracking	ACT §5 equality saturation; §5.4 inter-phase fallback

4.4 Hand-written reference program

The toy-ISA program for the reference matmul is written by hand *before* the generator exists, and becomes the comparison target for §12 and §13. Both source works benchmark generated output against hand-written references: TensorLift reports parity with hand-written Gemmini kernels at $1.014\times$ geometric mean (TensorLift §4.4, “Specification Completeness”); ACT compares against expert-written kernel libraries and production compilers. Note that ACT’s comparisons are against kernel *libraries*, not a hand-written reference artifact; the “hand-written reference” framing is TensorLift’s.

4.5 Second ISA: the transferability experiment

This is a scheduled deliverable, not a future-work note. v2 placed a second ISA under future work, which was the wrong shelf for it. The generator and the ISA description are authored by the same person, so a pair that has only ever run against the ISA it was designed for is self-referential: generality is *asserted*, never falsified. One independently authored ISA makes it measurable for a fraction of the cost of the parser, so it moves into the plan.

What varies. ISA-2 is not ISA-1 with renamed instructions. It differs along the four axes the pipeline is most likely to have quietly baked in:

- **Memory model.** Scratchpad plus accumulator, so a value can be live in a bank that is not the global space. ISA-1 is flat.
- **Compute primitive.** An outer-product / convolution-style unit rather than a square MAC tile, so the idiom matcher sees a different op signature and a different reduction structure.
- **Addressing.** 2-D strided DMA with an explicit offset pair rather than a 1-D pointer walk, so the structured-access descriptor is populated from different IR shapes.

- **Epilogue.** A clamp / saturation step — TensorLift Table 2 pass B5, `detect-clamp` — which ISA-1 deliberately excludes.

What must transfer unchanged. The claim is not that ISA-2 needs no new rules. It is that the *pipeline* transfers: the parser, the def-use graph, the recogniser’s interface (canonical descriptor in, `UNSUPPORTED` out), the annotation schema, the assembler driver and the emulator’s instruction dispatch are ISA-agnostic and are not edited. Everything ISA-specific lives in one schema file and one rule module.

The measurement. Two numbers are reported, and a negative result is a reportable result:

- **Transfer rate** — the fraction of pipeline stages that required no edit, counted per stage.
- **New-rule cost** — the number of matching rules and lines added for ISA-2, as a fraction of ISA-1’s, plus the locations of every edit forced outside the schema and rule modules.

That second location list is the falsification instrument: if the recogniser or the assembler has ISA-1 baked in, it names the line. This is the only falsifiable test of the word *declarative* in the title, and it is why the experiment is worth its half-day.

5 ttir extraction

5.1 Environment pinning

Fix and record the Triton version. The textual IR is not guaranteed stable across releases. Supporting evidence is direct: `triton-shared` pins an exact Triton commit in `triton-hash.txt` (e44bd1c83c1c3e8deac7c4f02683cfb3cc395c8b) and its own issue #345 is titled “Update triton to e44bd1c...”.

5.2 The GPU-free gate — VERIFIED

Extract `ttir` with an explicitly constructed target rather than relying on an active driver:

```
from triton.compiler import compile as tt_compile, ASTSource
from triton.backends.compiler import GPUPrimaryTarget

TARGET = GPUPrimaryTarget("cuda", 80, 32) # explicit: no driver, no GPU
compiled = tt_compile(
    ASTSource(fn=kernel, signature={...}, constexprs={...}),
    target=TARGET,
)
compiled.asm["ttir"] # also: ttgir, llir, ptx, cubin, source
```

Result on Triton 3.7.1: the gate passes. Measured line counts: vector-add 51, tiled matmul 160, matmul+bias+relu 174, modulo 67. Available `asm` keys: [`'cubin'`, `'llir'`, `'ptx'`, `'source'`, `'ttgir'`, `'ttir'`]. This is a go/no-go gate executed before any generator code is written; it has been executed and it passes.

5.3 Kernel corpus

Graduated so the pipeline is tested incrementally, mirroring TensorLift’s per-module reporting (its Table 3 separates PE, Execute/Load/Store Controllers, TensorGemm, TensorAlu and reports different reduction numbers for each).

Tier	Kernel	ttir	Verified structure
T0	vector add	51	tt.dot = 0, scf.for = 0 (VERIFIED)
T1	tiled matmul, single block	160	tt.dot = 1, scf.for = 1, iter_args = 1, scf.yield = 1 (VERIFIED)
T2	matmul + bias + ReLU epilogue	174	tt.dot = 1, scf.for = 1, tt.load = 3 (VERIFIED)
T3	modulo pointer wraparound (negative control)	67	arith.remsi = 1, scf.for = 0, tt.dot = 0 (VERIFIED)

Tier 3’s pattern is the one Triton-MTIA §8.1 identifies as needing rewriting, and the modulo survives into ttir as `arith.remsi`. It is therefore a genuine negative control. It must hit the opaque fallback cleanly, and confirming that is a required, documented test outcome, not a failure.

5.4 Fixture snapshotting

Persist extracted ttir for every corpus kernel immediately after successful extraction. All downstream work runs against these frozen fixtures rather than re-invoking the compiler. **State the limitation explicitly:** fixtures are a regression baseline, not a generality test. The parser is only ever exercised against one version’s output, and pinning the version conceals that.

5.5 Dumping ttgir as well

Also snapshot ttgir. It is not needed by the pipeline, but it documents the entity explosion that any future ML-side work would face, and it is free.

6 IR parsing and normalization

6.1 Hand-rolled parser: corrected justification

CORRECTED v1 justified a hand-rolled parser by claiming MLIR Python bindings “would reintroduce the exact build-from-source burden”. **This is false.** The stock triton wheel ships a working MLIR textual parser:

```
from triton._C.libtriton import ir
ctx = ir.context()
ir.load_dialects(ctx)                                # registers tt / arith / scf / math
mod = ir.parse_mlir_module("/tmp/T1_matmul.ttir", ctx) # -> module (parses)
```

Both the matmul and the modulo fixtures parse with zero additional build. **The conclusion nevertheless stands, for a different and verifiable reason: the shipped Python traversal API is too weak to walk a module.** operation has no walk; region has no get_block and is not subscriptable; a block cannot be reached from a parsed module. So the choice is between hand-rolling a text parser in Python and writing an out-of-tree C++ MLIR pass. Every reference implementation chose the latter; we choose the former for time, and we say so.

6.2 Def-use graph

Each SSA value is a node, operands are edges. The parser must not discard the structural information that §8 depends on.

Locations carry variable names, which v1 under-used. Measured: the Tier-1 fixture contains 169 `loc()` occurrences, and the locations are *named* — `loc("acc")`, `loc("pid_m")`, `loc("a_ptrs")`, `loc("c_ptrs")`, and function arguments such as `%x_ptr: !tt.ptr<f32> loc("x_ptr")`. Original Python identifiers are therefore recoverable for free, which is worth more than a line-count reduction: it makes every diagnostic legible, and it is the basis of the unsupported-operation report in §10.

This is the analogue of TensorLift’s decision to preserve structure — it keeps `scf.if` regions and named memrefs rather than lowering to branches and phi nodes (TensorLift §3.2), because “lowered into branches and phi nodes, erasing the if/else structure” destroys what downstream passes need.

6.3 Canonicalization: corrected scope

CORRECTED v1 proposed to “collapse redundant `tt.splat` → `tt.broadcast` chains and fold repeated `tt.addptr` accumulation into a single symbolic stride expression”, justified by analogy to TensorLift’s Phase A collapsing sign-extension chains. **Measured against the real Tier-1 fixture, the premise does not hold.**

Construct	Count	Assessment
<code>tt.splat</code>	13	Required broadcast expansion of a 2-D index computation
<code>tt.broadcast</code>	6	Required; each expands a size-1 axis to 64
<code>arith.muli</code>	10	Index arithmetic
<code>tt.addptr</code>	5	2 entry + 2 in-loop increments + 1 for the output pointer
<code>arith.extsi</code> / <code>arith.trunci</code>	0	No sign-extension analogue exists in ttir

The chain `rm[:,None]*sam + rk[None,:]*sak` is not redundant; it is how a broadcast index expression is formed. TensorLift’s A1/A2 collapse sign-extension and `trunci/ext` round-trips emitted by RTL synthesis — constructs with **no referent in ttir**. The analogy is therefore unsupported and the “IR reduction” metric of v1 would have measured near zero, inviting a false negative about the pass itself.

Corrected role. Canonicalization is *dropped* from the critical path. It is retained only as an optional preprocessing step with a different and defensible purpose: *vocabulary closure* for any future embedding-based work, where layout objects such as `#ttg.blocked<{sizePerThread=[1,1], threadsPerWarp=[1,32], warpsPerCTA=[4,1], order=[1,0]}>` must map to canonical entity identifiers before a vocabulary can converge across kernels. That purpose is out of scope here and is listed as future work. The §12 reduction metric is retained but is reported as informational, not as a success criterion.

7 Structured memory access recognition

7.1 Descriptor

A structured-access descriptor is `{base, sizes, strides, offsets, shape}`.

CORRECTED v1 described this as “base pointer, per-dimension stride, per-dimension shape — the same three-field addressing-attribute shape ... the same shape triton-shared’s pointer analysis attempts to recover (and sometimes fails to, per its own maintainers’ documented crashes)”. Two errors. First, the descriptor is not three fields: `tts.make_tptr` takes `base, sizes, strides, offsets, shape, static_strides, static_offsets, static_shape, order` (nine arguments), and the internal `PtrState` is `{offsets, sizes, strides}` plus a base. Second, `shape` in `tts.make_tptr` does *not* mean tile shape — it means the *wraparound boundary* of the address space, which is how

that project represents the modulo case. Our descriptor therefore adopts the same five semantic fields and is documented as a subset of `tts.make_tptr`, not as an independent invention.

Opportunity this creates. Because `tts.make_tptr` exists and is what Triton-MTIA itself consumes (Triton-MTIA §3.3; its Listing 2(d) emits `%desc = tts.make_tptr %x_ptr, ...`), the recogniser can be validated against it as a *conformance oracle* on the same fixtures, rather than only against hand-expected values. This is cheap and strengthens the evaluation materially. It also removes any claim to novelty for the descriptor itself, which is the honest position.

7.2 Bounded, syntax-driven walk — corrected operation list

Recognition is a bounded walk, not general symbolic pointer analysis. **CORRECTED** v1 listed the expected operation shapes as `tt.addptr` $\leftarrow$ `tt.splat`/`tt.broadcast` $\leftarrow$ `arith.muli`/`arith.addi` over `tt.make_range`/`tt.get_program_id`. **That list omits the case that the primary demonstration kernel actually depends on.** The measured structure of the Tier-1 loop is:

```
%c32_i32 = arith.constant 32 : i32
%acc_25:3 = scf.for %_k = %c0_i32 to %1 step %c1_i32
  iter_args(%a_ptrs_34 = %a_ptrs_14, %b_ptrs_35 = %b_ptrs_24, %acc_36 = %acc)
  -> (tensor<64x32x!tt.ptr<f32>>, tensor<32x64x!tt.ptr<f32>>, tensor<64x64xf32>) : i32 {
    %a = tt.load %a_ptrs_34 : tensor<64x32x!tt.ptr<f32>>
    %b = tt.load %b_ptrs_35 : tensor<32x64x!tt.ptr<f32>>
    %acc_37 = tt.dot %a, %b, %acc_36, inputPrecision = tf32
    %a_ptrs_38 = arith.muli %sak, %c32_i32 : i32
    %a_ptrs_39 = tt.splat %a_ptrs_38 : i32 -> tensor<64x32xi32>
    %a_ptrs_40 = tt.addptr %a_ptrs_34, %a_ptrs_39
    %b_ptrs_41 = arith.muli %sbk, %c32_i32 : i32
    %b_ptrs_43 = tt.addptr %b_ptrs_35, %b_ptrs_42
    scf.yield %a_ptrs_40, %b_ptrs_43, %acc_37
  }
```

The `tt.load` operands are **loop-carried** `scf.for` **iter-args**, advanced by a *constant* increment (`arith.muli %sak, %c32_i32`) and re-threaded through `scf.yield`. Neither `iter_args` nor `scf.yield` appeared in v1’s list, and `tt.expand_dims` — which occurs repeatedly and is essential to the index expression — is absent from it too. Without explicit handling of loop-carried pointer recurrences, §8’s precondition (“two operands each already resolved to a structured-access descriptor”) **fails on the very kernel it exists to demonstrate.**

The corrected walk surface: `tt.addptr`, `tt.splat`, `tt.broadcast`, `tt.expand_dims`, `arith.muli`, `arith.addi`, `tt.make_range`, `tt.get_program_id`, plus the two loop forms `scf.for` (iter-args) and `scf.yield`. A mitigating fact worth recording: the in-loop increment is a *constant*, not an affine function of the induction variable, so no affine/IV analysis is required — the recurrence is a simple loop-carried accumulation. This is why the bound is achievable at all, and it is stated so the difficulty is not underestimated.

Hop budget. From the `tt.load` operand to a terminal `tt.make_range` or `tt.get_program_id` the chain spans four to six edges in the Tier-1 fixture (iter-arg $\rightarrow$ entry value $\rightarrow$ `tt.addptr` $\rightarrow$ `tt.splat` $\rightarrow$ `arith.muli` $\rightarrow$ `tt.broadcast` $\rightarrow$ `arith.addi` $\rightarrow$ F). The budget is therefore set *empirically* from the fixtures and fixed at a stated value, not guessed.

7.3 Independent test

This pass is tested before idiom detection is attempted, against Tier 0 and Tier 3: Tier 0 must fully resolve; Tier 3 must cleanly fail to resolve. Both are asserted in automated tests.

8 Compute idiom detection

8.1 MAC / reduction idiom — VERIFIED

Match an `scf.for` whose `iter_args` include an accumulator tensor, whose body contains exactly one `tt.dot` consuming that accumulator, whose two operand loads resolve to structured-access descriptors, and whose `scf.yield` re-threads the accumulator. The structural skeleton quoted by v1 is confirmed against real compiled output, exactly:

```
%acc:N = scf.for ... iter_args(...) { %acc' = tt.dot %a, %b, %acc ...; scf.yield ... }
```

This is the inversion described in §2: TensorLift must build `scf.for` reductions from flattened bit-level arithmetic because RTL extraction destroys structure; Triton’s compiler emits them by construction, so the step is *recognition*. This is the load-bearing feasibility argument and it is confirmed empirically rather than assumed.

Precondition dependency, stated explicitly. This match’s correctness is conditional on §7.2 handling loop-carried iter-args. The two sections must be developed and tested together; v1 developed them independently, which is why the omission was invisible.

8.2 Attribute extraction

From the matched idiom, extract tile shape and dtype (the computational, α attributes) and the operands’ structured-access descriptors (the addressing, β attributes).

CORRECTED v1 contained an internal contradiction here. §6.2 said the α attributes are *extracted from the matched idiom*; §7.1 said they are “fixed at schema-design time in §2.2”. Both cannot be the source of truth. The corrected position: the ISA fixes the set of *admissible* tile shapes and dtypes (that is the ISA’s α vocabulary), and the matcher *must verify* that the idiom’s tile shape and dtype belong to that set, failing to the opaque fallback otherwise. Verification against a declared vocabulary is the operation that makes §9 a lookup rather than a search, and it is also what makes a second ISA meaningful.

8.3 Elementwise epilogue idiom

A fixed small set of post-loop elementwise operations applied to the accumulator before the final `tt.store`: `arith.addf` against a broadcast bias, or a max-against-zero ReLU pattern. Implemented as a second, independent rule rather than folded into the MAC rule, so future idioms are additive rather than requiring the MAC matcher to be rewritten. This directly follows TensorLift: “passes annotate rather than rewrite” (TensorLift §3.2).

8.4 Annotation

Each matched idiom is tagged on the graph with a metadata attribute analogous to TensorLift’s `tensorlift.mac` and `taidl.*` annotations (e.g. `toyisa.mac`, `toyisa.dma_role`), so that assembly is a pure consumer of annotations.

CORRECTED v1 attributed the same annotate/assemble separation to “TensorReduce and TAIDL/ACT”. `TensorReduce` appears exactly once in v1, carries no citation, and is not one of the four source papers. The reference is removed. The separation is attributed to TensorLift and ACT, both of which are verified.

9 Backend assembly

9.1 Two paths

Assembly splits into a compute path (annotated MAC and elementwise idioms $\rightarrow$ instruction templates, filling in the α attributes verified in §8) and a DMA path (annotated structured-access descriptors $\rightarrow$ load/store instructions, filling in base/sizes/strides/offsets). This is the two-path split ACT and TensorLift’s Stage 3 both use (TensorLift §3.3 is “Stage 3: TAIDL Assembly”).

Now a selection, not a lookup. With §4.2 supplying two DMA and two MAC variants with costs, §9 selects the minimum-cost variant consistent with the verified α attributes. This is ACT §7’s candidate-selection loop applied at single-instruction granularity, and it is what entitles the artifact to be called a generator.

9.2 Ordering

CORRECTED v1 claimed ordering “is recovered directly from the def-use graph’s topological order” and that because “Triton IR already is a dataflow graph, ... this ordering is free”. **Overstated.** A flat topological sort over the def-use graph *flattens the region hierarchy* and will mis-order the kernel: both `tt.loads` must remain inside the `scf.for`, `scf.yield` must terminate the loop body, and the accumulator is loop-carried. Ordering is *derivable*, but requires region-aware traversal that respects nesting. The corrected statement: ordering follows from a region-aware traversal of the def-use graph, and no separate FSM-inference step is needed — *unlike* TensorLift §3.3, which recovers ordering constraints by analysing FSM register updates across instruction groups “because RTL exposes no explicit dataflow graph”. The contrast with TensorLift is valid and is the point; the word “free” is not.

9.3 Emission

The final instruction stream is a linear list of opcode plus resolved operands, together with a parallel human-readable disassembly for debugging and for the completeness report.

10 Opaque fallback

Any operation reaching assembly without a `toyisa.*` annotation is emitted as an explicit `UNSUPPORTED(<op name>)` marker rather than being dropped, mistranslated, or causing a crash. This adopts TensorLift’s principle verbatim — “An instruction that fails to match a template simply receives no annotation, and Stage 3 falls back to opaque semantics rather than producing incorrect TAIDL” (TensorLift §3.2) — and ACT’s soundness requirement that a generator must never silently miscompile (ACT §3.3).

10.1 Correction: a parse-failure path is also required

v1 had no parse-failure path. §8’s mechanism covers *semantic* non-match only. Nothing covered `ttir` that the hand-rolled tokenizer cannot parse: nested attribute dictionaries, multi-result forms such as `%acc_25:3`, region bodies, layout-typed type strings, and the `#loc` definition table. Because §13 requires Tier 3 and the fuzz kernel to fail *cleanly*, a sibling route is mandatory:

<code>UNSUPPORTED(<op name>)</code>	<code># semantic: recognised op, but no lowering</code>
<code>PARSE_UNSUPPORTED(<line>:<col>)</code>	<code># syntactic: tokenizer could not consume input</code>

Both are first-class, tested outcomes. Because §6 established that locations carry original variable names, the parse-failure report can cite the originating Python identifier, which turns a failure into a diagnostic rather than a dead end.

11 PyTorch integration

This is the section that makes the artifact a contribution rather than an exercise.

11.1 Registration

```
from torch._dynamo.backends.registry import register_backend
from torch._dynamo.device_interface import DeviceInterface, register_interface_for_device

def toyisa(gm, example_inputs):
    # gm -> toy ISA instruction stream -> device emulator
    return gm.forward

class ToyISAInterface(DeviceInterface):
    # stub-level implementation, modelled on torch_openreg/compiler.py
    ...

register_backend(name="toyisa")(toyisa)
register_interface_for_device("toyisa", ToyISAInterface)
```

A real PyTorch operation is then compiled and executed through the generated backend on the toy device. The reference pattern is PyTorch’s own in-tree `OpenReg`, which PyTorch explicitly describes as stub-level and which emulates its device on the CPU.

11.2 What this demonstrates

Without §11 the artifact demonstrates that `ttir` can be lowered to toy assembly — interesting, but not connected to anything. With it, the artifact demonstrates that a declarative ISA description yields a `torch.compile` backend that PyTorch will dispatch to. That is the claim of §1, and it is the difference between a demonstration and a demo.

11.3 Scope

`DeviceInterface` spans 30 method slots across four nested classes (`device`, `Event`, `Stream`, `Worker`); most have usable defaults and a stub-level implementation suffices. We do *not* claim to implement the device-integration surface fully — no serialisation, no distributed backend, no profiler plugin, no autocast. This is a declared limitation, not an oversight, and it is listed in §16.

12 Benchmarking and coverage

Coverage is the headline metric, not an appendix. v1 treated §9.1 as one metric among several. Given §3, coverage is the question that decides whether a vendor’s model runs at all — which is exactly what Triton-MTIA’s Table 1 reports (94.5% inference / 94.7% training) and what ACT’s 2.3× NKI coverage claim measures (ACT §8.3–§8.6). The evaluation is re-centred accordingly.

12.1 Coverage — corrected definition

CORRECTED v1 defined coverage as “the fraction of `ttir` op nodes that received a `toyisa.*` annotation”. **This metric is gameable and semantically weak:** one unannotated operation early in a dataflow chain renders every downstream annotation meaningless, while the node percentage stays high. A blended percentage would therefore overstate real coverage. Corrected definitions, all reported *per tier*:

Metric	Definition
Kernel fully lowered	Boolean. True iff every <i>value-producing</i> operation is either annotated or provably elided. Reported before any percentage.
Largest lowered subgraph	Size of the largest connected subgraph of annotated operations, as a fraction of the dataflow-relevant graph.
Annotated node fraction	Retained for comparability, explicitly labelled as an upper bound.
Unsupported inventory	The list of <code>UNSUPPORTED</code> and <code>PARSE_UNSUPPORTED</code> markers, each with its originating variable name from <code>loc</code> .

Reporting is per tier *and per ISA*, never merged into a single number, mirroring TensorLift’s per-module reporting (TensorLift Table 3 covers PE, Execute/Load/Store Controllers, TensorGemm, TensorAlu and GenVME separately, with different reduction figures for each).

12.2 Selection quality

With ISA variants (§4.2), report the cost of the selected lowering against (a) the hand-written reference program and (b) an oracle minimum obtained by exhaustive enumeration of valid lowerings. This is the metric that demonstrates the generator is choosing rather than templating, and it strictly supersedes v1’s “reference-match” metric, which required “same instruction count and shape” and would have flagged legitimate alternative lowerings as failures — a flaw v1 conceded in the same paragraph that specified the metric.

12.3 IR reduction — informational only

Raw `ttir` node count, post-canonicalisation node count, final instruction count, per kernel. Reported for completeness. **CORRECTED** v1 framed this as a success criterion on the analogue of TensorLift’s 92.9% / 41.2% figures. Two corrections: the reduction is expected to be near zero because the §6.3 premise does not hold, and the citation was wrong — TensorLift has no §9. The figures are in TensorLift §4.2, Figure 4 and Table 3, and the numbers themselves (24.8% Gemmini, 41.2% VTA, up to 92.9% on a processing element) are correct.

12.4 Reference-match

Comparison against the hand-written toy-ISA program of §4, with structural equivalence defined *semantically* — the same DMA descriptor set, MAC operation and epilogue, order-independent up to dataflow dependencies — rather than by instruction count. Numerical equivalence is established by §14.

12.5 Generation latency

Wall-clock from `ttir` input to emitted program, per kernel, framed as a one-time per-kernel-shape cost rather than a per-compilation runtime cost, following the framing TensorLift uses for its pipeline cost (TensorLift §4.1, “Pipeline cost”; v1 cited §4.2, which is “Semantic Lifting Effectiveness” — an error in v1).

12.6 Cross-ISA transfer

The headline result of §4.5, reported as one row per pipeline stage and one column per ISA: whether the stage ran unedited on the second ISA, and the edit forced if it did not. Kernels, coverage and selection quality are all reported *per ISA*; a figure merged across the two would conceal exactly what the experiment exists to expose. The summary is the transfer rate, the

new-rule cost, and the list of every edit that landed outside the schema and rule modules — that list, not the percentage, is the falsification instrument.

12.7 Results table first

All of the above is assembled into a single results table before any narrative is written. The table is the artifact; prose is generated from it.

13 Testing strategy

Testing is layered to match the pipeline’s stages, so a failure localises to one component instead of surfacing as “the end-to-end demo is wrong”. This is v1’s strongest section and is retained largely unchanged, with the named additions.

1. **Fixture-based regression.** Every corpus kernel’s frozen `ttir` is a checked-in fixture; parsing, recognition and idiom detection each have unit tests against fixtures, independent of a live Triton install. Limitation stated: this is a regression baseline, not a generality test.
2. **Canonicalisation.** Re-scoped. Since §6.3 is off the critical path, the test asserts idempotence and vocabulary-closure properties only, and does *not* assert a reduction magnitude that the data says will not occur.
3. **Memory-access recognition.** Tier 0/1/2 resolve to the expected descriptors with exact `base/sizes/strides/offsets/shape` values, not merely “resolved: yes/no”; Tier 3 resolves to unstructured. **Added:** conformance against `tts.make_tptr` on the same fixtures (§7).
4. **Idiom detection.** The MAC pattern is found exactly once in Tier 1/2 with the expected operand bindings; and it is *not* falsely triggered on Tier 0, which contains no `tt.dot` at all (**VERIFIED**). A true-negative test is as important as the true-positive ones.
5. **Loop-recurrence test. New, and mandatory.** Because §7.2 is the corrected failure mode, a dedicated test asserts that in-loop `tt.load` operands carried through `scf.for` iter-args and re-threaded by `scf.yield` resolve to the correct descriptor. This test exists because v1’s omission of that case would have passed every other test in this list while failing on the primary kernel.
6. **End-to-end integration.** For each tier, run the full pipeline and assert (a) the expected coverage outcome of §12, (b) that Tier 3 produces at least one `UNSUPPORTED` marker and zero crashes, and (c) that Tiers 1–2’s generated streams execute against the device emulator without runtime errors.
7. **PyTorch integration test. New.** Assert that a real PyTorch operation compiles through the registered backend and produces numerically correct results on the toy device (§11). Without this, §11 is untested code.
8. **Transfer test. New, and the point of §4.5.** Run the unedited pipeline against the second ISA’s schema and corpus, and record the transfer rate, the new-rule cost and every forced edit. An expectation that some stages will not transfer is not a licence to skip the measurement; the expectation is the hypothesis and the measurement is the result.
9. **Numerical differential testing.** Execute the generated program on randomised inputs through the emulator and compare against eager PyTorch execution, across a small matrix of shapes and dtypes chosen to stress the matcher. See §14 for the precision constraint, which is *mandatory* rather than optional.
10. **Canonicalisation idempotence.** Running the pass twice equals running it once.

11. **Negative / fuzz pass.** At least one kernel outside the corpus, written last, intended to break assumptions in §7–§8 (a non-power-of-two block size; a no-op elementwise identity inserted mid-graph). Not expected to pass; the purpose is to confirm the failure mode is the opaque fallback or `PARSE_UNSUPPORTED`, not a crash or silent wrong output.

13.1 Unified test, benchmark and CI protocol

A four-person team tests the same artifact, so the layer list above is necessary but not sufficient: four people can satisfy it and still disagree about what the expected number is. **CORRECTED** The most consequential defect found while laying out the plan was not in the pipeline at all — it was that the generated IR embeds the *absolute source path* of the file that produced it:

```
#loc=loc("/home/dev/proj/verify/dump.py":31:0)
```

Two people generating the same kernel therefore produce *different bytes*, and every hash, diff and golden-file comparison fails for whoever did not generate it first. This was measured on the real corpus before normalisation and is the reason the protocol below exists. The canon is therefore: `fixtures/raw/` as dumped, `fixtures/*.ttir` with `loc("<abs path>")` rewritten to `loc("LOCFILE")`, and `fixtures/GOLDEN.json` carrying two blocks with different edit rules: a machine-derived `observations` block that no human may edit, and a reviewed `intent` block stating what each tier was designed to exhibit. `make golden-check` fails, naming the field, if the two ever disagree.

These rules are properties of the method, not tooling preferences. Full text: `docs/team/testing-ci.md`.

1. **No expectation is typed by hand.** Every expected value comes from a fixture or from `GOLDEN.json`. Counts such as “13 splats” are observations and belong in the canon; only the reviewed `intent` block licenses a literal in a test.
2. **The layer is fixed by the directory**, and the marker is applied automatically, so a test cannot be filed in the wrong layer by forgetting to annotate it. Unit and contract layers carry the load; benchmark is a fourth layer that publishes and never gates.
3. **Every contract test is parametrised over two input classes:** `handbuilt` (built as a literal, runs from hour 0) and `real` (consumes the real upstream stage). Until the upstream exists the `real` row is an `xfail`; one flag, flipped at the end of day 3, converts every such row into a hard failure. This is what makes four-way parallel work safe *and* keeps the parallelism from becoming permanent.
4. **Module and contract coverage is checked, not trusted:** every module under `src/triton_toyisa` must have `tests/unit/test_<module>.py`, and every contract file must have `tests/contract/test_<name>.py`. The same checker enforces that `triton` is imported nowhere outside `harness/`, which is why the suite runs with no GPU, Triton or torch.
5. **Determinism is a rule, not an aspiration:** no GPU, no network, no timing assertions, one declared seed, LF endings, and a fixture-refresh job that is manual because the fixtures are an artifact of one pinned Triton version.
6. **The benchmark emits every row every run.** A metric that cannot be computed is recorded as `unavailable` with a reason rather than omitted, so the published table cannot quietly lose a metric; the report is generated from that table rather than written alongside it.

The protocol is instantiated, not described: the canon is generated, its contract test passes (30 assertions over 4 fixtures and 182 operations), and the benchmark runner emits its 32 rows today with the pipeline stubbed — so the first real number is a diff against a known baseline, not a new file.

14 Numerical validation harness

14.1 Device emulator

The toy ISA is directly interpretable: DMA load/store map to array slicing with the resolved descriptor; MAC maps to `matmul/tensordot` over the verified tile shape; elementwise maps to the corresponding ufunc. **Role corrected:** this is the *device emulator* behind the PyTorch backend of §11 — the same role `libopenreg.so` plays for PyTorch’s own reference backend — not a substitute for the backend.

The emulator consumes the same instruction stream a real simulator would, which keeps validation honest: it tests the generated program, not a shortcut around it.

14.2 Precision: a mandatory constraint, not a footnote

CORRECTED v1 mentioned tolerance only to say “record the tolerance used”. The frozen fixture makes this insufficient: the Tier-1 `tt.dot` carries `inputPrecision = tf32` as a literal attribute of the IR. A NumPy `float32` emulator **cannot** match torch/Triton numerics, because the `tf32` path truncates mantissa bits on the multiply inputs. Therefore:

- The emulator must reproduce the declared `inputPrecision` explicitly, or
- the tolerance must be derived from the precision difference and stated per dtype, with the derivation recorded; and
- the *accumulation order* declared by the ISA (§4.2) is the order the emulator reduces in, so a residual mismatch is attributable to precision and not to reassociation of a non-associative operation.

This is not hypothetical; it is a property of the input IR and it is knowable before the emulator is written. Triton-MTIA independently reports mixed-precision handling as a source of out-of-tolerance results requiring explicit casting (Triton-MTIA §8.1), which corroborates that per-dtype tolerance policy is the correct treatment.

Parity is reported as exact match for integer and low-precision paths and as within-tolerance match for float paths, with the tolerance recorded. Both source works are explicit about their own verification bounds — TensorLift proves core compute and data-movement semantics via Z3 SMT and validates the remainder against golden simulator data; ACT states soundness modulo the foundational axioms of XLA-HLO IR — and we follow that practice.

15 Schedule

CORRECTED v1 invoked a “5-day budget” four times and never laid it out. A schedule is a deliverable, because the plan has a known critical path and an unmitigated parser risk. Ordering is chosen so that a truncated project still ships a working artifact.

Block	Deliverable	Gate / risk
0 (2–3 h)	Correct §2.1, §6.1, §6.3, §7.2; re-centre coverage	Prevention. These are the four v1 defects that would be hit during the build, not after it.
1 (0.5 d)	Extraction gate, fixtures for all four tiers, <code>ttgir</code> snapshot	VERIFIED already. Zero risk.
1a (0.25 d)	Fixture canon: normalisation of <code>loc</code> source paths, <code>GOLDEN.json</code> , <code>make golden-check</code> , <code>Makefile</code> , CI workflow, bench runner	Done. The test canon and the 32-row benchmark exist and run before any pipeline code, so everyone builds against one set of numbers from hour 0.
1b (0.25 d)	Seam smoke test: <code>register_backend</code> + a <code>DeviceInterface</code> stub that lowers one hard-coded kernel and runs a real <code>torch</code> op end to end	De-risks the highest-value block by proving the interface on day 1 rather than day 6.
2 (1.5 d)	Parser $\rightarrow$ def-use graph; structured-access recogniser <i>including</i> <code>iter-args</code> , <code>scf.yield</code> , <code>expand_dims</code> ; Tier 0 must resolve, Tier 3 must not	Critical path. The hand-rolled parser is the largest single risk. If block 2 overruns, cut block 5.
3 (1 d)	MAC and epilogue detectors; annotation; true-negative test on Tier 0; loop-recurrence test	Mechanical once block 2 is correct.
4 (1 d)	Assembly with variant selection and costs; device emulator; <code>PARSE_UNSUPPORTED</code> path	Medium. Ordering must be region-aware.
5 (0.75 d)	<code>register_backend</code> + <code>DeviceInterface</code> stub; run a real <code>torch</code> op through the backend	Highest value per hour in the plan. Template exists in-tree.
6 (0.5 d)	Coverage and selection-quality tables; limitations document	Cheap, and it is the artifact a reader actually reads.
7 (0.5 d)	Second ISA (§4.5): its schema, one rule module, and the transfer and new-rule-cost measurement	The falsifiable test of the word <i>declarative</i> . A negative result is reported, not hidden.

The honest total is about 6.75 days, not 5. v1 invoked a “5-day budget” and never added it up; v2 listed the blocks but stopped at 5.25 days and left the second ISA outside the plan entirely.

This revision adds block 1a, the test canon and benchmark, which is a quarter-day. Summing the rows above gives 6.25 days before block 0’s two to three hours, and with the same integration slack as before, roughly 6.75. Saying so is part of the correction: a budget that is only asserted is not a budget.

Cut order under time pressure: fuzz kernel → canonicalisation entirely → block 7 (second ISA) → Tier 2 epilogue → Tier 3 fixture. **Never cut:** the Tier 3 negative control, the coverage table, the true-negative test, and §11. When the cap is hard, the order tells the reader what was dropped and the limitations document records it.

16 Documentation of scope boundaries

A running “known limitations” list is maintained alongside the code, generated from every explicit non-goal in §4–§11, and cross-referenced against which source work solves it at full generality. It is a deliverable, not an apology: it is what lets the project be presented honestly as a demonstration of the automated-generator layer rather than as a competitor to ACT’s generality.

Limitation	Who solves it at full generality
Bounded-hop pointer walk; no general symbolic pointer analysis	triton-shared’s structured analysis (<code>tts.make_tptr</code>) handles more, and Triton-MTIA <i>reuses</i> it; full generality would require ACT-style handling of addressing attributes. Note the corrected basis for this bound: it is a time budget, not a documented crash.
Tiling not synthesised; inherited from the source IR	ACT §3.1 tiles before its backend runs; we start downstream, deliberately.
Greedy selection, no backtracking	ACT §5 equality saturation and §5.4 intra-phase fallback. Note precisely: ACT’s <i>addressing</i> phase is integer constraint programming (§6); equality saturation is the <i>computational</i> -attribute phase (§5). v1 conflated these.
Two ISA vocabularies; transfer measured, not asserted	§4.5 runs the unedited pipeline against an independently authored second ISA and reports the transfer rate and the new-rule cost. Generality beyond two designs remains open; the number is offered as evidence, not as a claim.
Device integration implemented at stub level only	PyTorch’s own <code>OpenReg</code> reference is also stub-level, by design.
Operator coverage measured on four kernels, two ISA variants	Triton-MTIA Table 1 (2,590 ATen schemas, ~540 Core ATen IR); ACT §8.3–§8.6 (six platforms, 2.3× NKI coverage).
Python text tool; not an MLIR pass	ACT, triton-shared, triton-linalg, intel-xpu and Triton-MTIA are all C++ MLIR passes. Ours cannot be dropped into Triton’s pipeline.

16.1 Future work, ordered

A second ISA stood first in this list in v2. It is now §4.5 and schedule block 7, because the cheapest credibility gain available is not future work.

1. **Reuse `tts.make_tptr` semantics directly** rather than as a conformance oracle, or state explicitly why a bounded Python walk is preferable to the analysis Triton-MTIA consumes in production.

2. **Canonicalisation for vocabulary closure**, as a prerequisite for any embedding-based cost model.
3. **A learned cost model** to replace the analytic per-instruction cost of §4.2. ACT makes this explicit: “Performance statistics can be collected using different fidelity cost models, including cycle-accurate simulators, analytical models, and learned cost models. Note that the enumeration pipeline and the ACT algorithm are agnostic to the choice of cost model” (ACT §7). The analytic model of §9 is retained as the baseline to beat.

A Evidence appendix

Every quantitative statement in this document traces to one of the following. Reproduction commands are given where applicable.

Assertion	Evidence
GPU-free <code>ttir</code> extraction works	<code>triton.compile(ASTSource(...), target=GPUPort("cuda",80,32))</code> on Triton 3.7.1; <code>asm</code> keys <code>['cubin','llir','ptx','source','ttgir','ttir']</code>
Tier-1 MAC structure	Exactly one <code>scf.for</code> , one <code>iter_args</code> , one <code>tt.dot</code> , one <code>scf.yield</code> in the 160-line fixture
In-loop operands are <code>iter_args</code>	<code>tt.load %a_ptrs_34</code> where <code>%a_ptrs_34</code> is a loop-carried <code>iter_args</code> value advanced by the constant <code>arith.muli %sak, %c32_i32</code>
Splat/broadcast chains are not redundant	13 <code>tt.splat</code> , 6 <code>tt.broadcast</code> , 10 <code>arith.muli</code> , 5 <code>tt.addptr</code> , and 0 <code>arith.extsi/arith.trunci</code>
Locations carry variable names	169 <code>loc(</code> occurrences; <code>loc("a_ptrs")</code> , <code>loc("pid_m")</code> , <code>loc("x_ptr")</code>
Tier-3 modulo reaches <code>ttir</code>	<code>arith.remsi = 1</code> in the 67-line fixture
MLIR bindings ship with the wheel	<code>ir.parse_mlir_module + ir.load_dialects</code> parse both fixtures; traversal API lacks <code>walk/get_block</code>
TF32 in the IR	<code>tt.dot %a, %b, %acc_36, inputPrecision = tf32</code>
TensorLift pass taxonomy	Table 2: A1 canon-bitmanip, A2 narrow-types, B3 detect-mac, B4 specialize-control, B5 detect-clamp, C6 reconstruct-loops, C7 lift-to-linalg, D8 emit-taidl-metadata
TensorLift section map	§3.1/3.2/3.3, §4.1 Experimental Setup, §4.2 Semantic Lifting Effectiveness, §4.3 Correctness Verification, §4.4 Specification Completeness, §4.5 End-to-End, §5 Related Work. There is no §9.
ACT phase split	§5 Phase 1 instruction selection via equality saturation; §6 Phase 2 memory allocation via integer constraint programming; §7 cost models, agnostic; §8.3–§8.6 comparative evaluations
ACT cost-model signature	<code>cost = cost_model(asm)</code> (Fig. 18(a) line 7) — the input is emitted assembly
ACT analytical cost model	“uses a simple analytical cost model based on individual instruction costs” (§8.5 ACT-AMX, §8.6 ACT-Gemmini)

Assertion	Evidence
TAIDL API	<code>add_data_model(model_name, dimensions, unit_dim, var_type);</code> <code>add_instruction(instruction, computation_attr,</code> <code>addressing_attr, cost, update, constraints);</code> <code>add_semantics(semantics);</code> ANTLR4 grammar under <code>taidl/antlr4/</code>
triton-shared status and limits	README line 1 “no longer maintained”; last commit 2025-12-05 (#367); README on structured-only prototype and “early stage”; <code>TritonToUnstructured</code> 595 lines; <code>UnstructuredToMemref</code> 422 lines; <code>PtrAnalysis.h</code> documents % <code>mod_offset</code> handling; <code>triton_shared.cc</code> is 8 lines; two parallel <code>PtrAnalysis</code> implementations (1,375 and 1,959 lines)
triton-shared commits to <code>tts.make_tptr</code>	<code>TritonStructuredDialect.td:34 TTS_Op<"make_tptr"></code> with nine arguments; consumed by Triton-MTIA (its Listing 2(d))
PyTorch surfaces	<code>native_functions.yaml</code> - <code>func:</code> = 2,590; <code>register_lowering()</code> in <code>lowering.py</code> = 209 (265 across <code>torch/_inductor</code>); <code>register_decomposition()</code> in <code>torch/_decomp</code> = 186
PyTorch integration APIs	<code>register_backend(compiler_fn=None, name=None, tags=());</code> <code>DeviceInterface</code> at <code>torch/_dynamo/device_interface.py</code> (line 40), 30 method slots, 4 nested classes; <code>register_interface_for_device</code>
OpenReg scope	In-tree README: “not to implement a fully functional, high-performance PyTorch backend, but to serve as a minimalist reference implementation for mechanism verification”; <code>libopenreg.so</code> “uses the CPU to emulate a CUDA-like device”; <code>torch_openreg/compiler.py</code> registers both a backend and a <code>DeviceInterface</code>
Triton-MTIA production numbers	Table 1: 94.5% inference / 94.7% training success; Inductor kernels = 50% of layers and 47% of non-GEMM execution time; 60 model types; §8.1 modulo pointer arithmetic among 5 kernels requiring rewriting; §5.1 upstream Inductor codegen tailoring to avoid modulo and division

B Corrections register: v1 → v2

Every claim in v1 that failed verification, with its disposition. This appendix exists so the revision is auditable rather than silent.

v1 location	Defect	Disposition
§1.4	triton-shared “crashes on unstructured access”, “monolithic hard-to-extend pass”, “cited directly by its own maintainers”	CORRECTED §2.1. Unsupported, and the repository is archived.
§1.4 / §12.2	“arbitrary pointer arithmetic: triton-shared attempts this and documents its own failures”	CORRECTED. It has a tested unstructured path and an explicit modulo representation.
§4.1	MLIR bindings “would reintroduce the exact build-from-source burden”	CORRECTED §6.1. They ship and parse; the real obstacle is the traversal API.

v1 location	Defect	Disposition
§4.3	“redundant <code>tt.splat</code> / <code>tt.broadcast</code> chains”; reduction as a success criterion	CORRECTED §6.3. Premise falsified by measurement; genuinely 0 sign-ext/trunc in <code>ttir</code> .
§4.3	“TensorLift reports its 92.9%/41.2% figures (§9.2)”	CORRECTED . TensorLift has no §9; figures are in §4.2/Fig. 4/Table 3. Numbers were right.
§5.1	Descriptor described as “three-field” and as what triton-shared “attempts to recover (and sometimes fails to)”	CORRECTED §7. Nine-argument op; five semantic fields adopted; novelty disclaimed.
§5.2	Operation list omits <code>scf.for</code> iter-args, <code>scf.yield</code> , <code>tt.expand_dims</code>	CORRECTED §7.2. Would have failed on the primary kernel.
§6.1	Precondition assumed independent of §5	CORRECTED §8. Dependency now explicit; dedicated test added.
§7.1 vs §6.2	α attributes “fixed at schema-design time” versus “extracted from the matched idiom”	CORRECTED §8. Now: verify against a declared admissible set.
§7.2	“Triton IR already is a dataflow graph, so this ordering is free”	CORRECTED §9. Region-aware traversal required; <code>scf.yield</code> terminates; accumulator is loop-carried.
§2.2 / §2.4	One DMA, one MAC, one elementwise; ISA described as TAIDL’s “miniature”	CORRECTED §4.2. Alternatives and costs added; TAIDL fidelity claim narrowed honestly.
§6.4	“TensorReduce and TAIDL/ACT”	REMOVED . Orphan citation, not a source paper.
§8	No parse-failure path	ADDED §10.1. <code>PARSE_UNSUPPORTED</code> is now first-class.
§9.1	Coverage = annotated node fraction	CORRECTED §12. Gameable; boolean and subgraph metrics added.
§9.2	“TensorLift §4.2/Table 3” for reduction; reported as success criterion	PARTLY CORRECTED . Citation verified correct; role downgraded to informational.
§9.3	Structural equivalence = “same instruction count and shape”	CORRECTED §12. Semantic equivalence defined; count-based test would reject valid lowerings.
§9.4	“TensorLift ... (§4.2, ‘Pipeline cost’)”	CORRECTED . “Pipeline cost” is in §4.1.
§10.4 / §11.3	tf32 treated as a footnote	CORRECTED §14.2. It is a literal in the IR; a float32 emulator cannot match.
§10.4 vs §3.3	“fixed-shape kernel” versus testing non-square and alternate block sizes	CLARIFIED . Block shape (ISA) versus problem shape (kernel) now distinguished.

v1 location	Defect	Disposition
§12.2	“full generality would require ACT-style equality saturation over addressing expressions”	CORRECTED. Addressing is integer constraint programming (ACT §6); equality saturation is the α phase (ACT §5).
§1.5 / §12	No schedule; generality never tested	ADDED §15; second ISA promoted to the first item of future work.
—	PyTorch integration absent entirely	ADDED §11. The highest-value addition in v2.

Summary of v2. The plan was sound; its *justifications* were partly not. v2 keeps every load-bearing design decision of v1 — recognition over reconstruction, bounded pointer walk, annotate rather than rewrite, opaque fallback, layered testing — while removing the three false justifications, repairing the two component specifications that would have failed on the primary kernel, re-centring the evaluation on coverage, and aiming the artifact at the integration surface that PyTorch actually provides. The narrow claim of §1 is the claim v2 is entitled to make.